

FRONTEND TRAINING NOTE

Frontend Authentication, Forms & HTTP Methods Fundamentals

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

WHAT THIS GUIDE COVERS

HTML Forms — structure, elements, and purpose

Login & Register Pages — design, validation, and best practices

Authentication vs Authorization — clearly explained

Basic Website Security — passwords, HTTPS, cookies, sessions, tokens

How Login Systems Work — step-by-step flow

HTTP Methods — GET, POST, PUT, DELETE, PATCH

JavaScript Validation — practical snippets

Quiz, Mini Tasks & Summary

Table of Contents

Table of Contents

Table of Contents	2
1. Introduction to Forms	4
1.1 Why Forms Are Important.....	4
1.2 Common Types of Forms	4
1.3 Common HTML Form Elements	4
1.4 Simple HTML Form Example.....	5
2. Login Pages.....	6
2.1 What Happens When a User Clicks Login?	6
2.2 Simple HTML Login Form.....	6
2.3 JavaScript Login Validation	7
3. Register Pages	8
3.1 Common Registration Fields	8
3.2 Simple HTML Registration Form.....	8
3.3 JavaScript Registration Validation	9
4. Authentication vs Authorization	10
4.1 Authentication — "Who are you?"	10
4.2 Authorization — "What can you access?"	10
4.3 Comparison Table	10
5. Basic Website Security Concepts	11
5.1 Core Security Concepts.....	11
5.2 Show / Hide Password — Simple JS Example	12
6. How Login Systems Work — Step by Step	13
6.1 The Full Login Flow	13
6.2 Sessions vs Tokens.....	13
7. HTTP Methods.....	14
7.1 GET — Request Data.....	14
7.2 POST — Send Data Securely	14
7.3 PUT — Update Existing Data	15
7.4 PATCH — Partial Update	15
7.5 DELETE — Remove Data	15
7.6 HTTP Methods Comparison Table	15
8. Frontend Validation Using JavaScript	16
8.1 Empty Field Check	16
8.2 Email Format Validation	16

8.3 Password Length Check.....	16
8.4 Confirm Password Match.....	16
8.5 Reusable Error Display Function	17
9. Best Practices for Login & Register Pages.....	18
10. Practical Mini Tasks	19
Task 1 — Build a Login Form (HTML Only)	19
Task 2 — Add JavaScript Validation to Your Login Form	19
Task 3 — Design a Complete Registration Page.....	20
11. Quiz	21
Part A — Multiple Choice (Circle the correct answer)	21
Part B — Short Answer	22
12. Chapter Summary.....	23

1. Introduction to Forms

A form is an HTML element that collects input from users and sends it somewhere — usually to a server. Almost every website you use today relies on forms: from searching on Google, to logging into your email, to filling in a contact page.

1.1 Why Forms Are Important

- ✓ They allow users to communicate with a website.
- ✓ They collect data: names, emails, passwords, messages, and more.
- ✓ They trigger actions: login, registration, payments, bookings.
- ✓ Without forms, websites would be one-way — you could only read, never interact.

1.2 Common Types of Forms

Form Type	Purpose
Login Form	Allows a returning user to access their account.
Register Form	Collects details to create a new user account.
Contact Form	Lets visitors send a message to the website owner.
Feedback Form	Collects opinions, ratings, or survey responses.
Search Form	Accepts a keyword and returns matching results.
Checkout Form	Collects payment and shipping details in e-commerce.

1.3 Common HTML Form Elements

Element	Description	Example Use
<input>	Single-line text entry (text, email, password, checkbox, radio, etc.)	Name, email, password fields
<label>	Describes what an input is for — important for accessibility.	"Email Address"
<textarea>	Multi-line text entry.	Comment or message boxes
<button>	A clickable element that submits the form or triggers an action.	Submit, Login, Send
<select>	A dropdown list of options the user can choose from.	Country, gender, category
<checkbox>	Allows users to tick one or more options.	"I agree to terms"
<radio>	Allows users to choose only ONE option from a group.	Male / Female / Other

1.4 Simple HTML Form Example

```
<!-- Basic Contact Form -->
<form action="submit.php" method="POST">

  <label for="name">Your Name:</label>
  <input type="text" id="name" name="name" placeholder="Enter your name" required>

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" placeholder="you@example.com"
required>

  <label for="message">Message:</label>
  <textarea id="message" name="message" rows="4" placeholder="Type
here..."></textarea>

  <button type="submit">Send Message</button>

</form>
```

KEY RULE:

Always pair every `<input>` with a `<label>`. This makes forms accessible to screen readers and improves usability for all users, including those on mobile devices.

2. Login Pages

A login page is the gateway to a user's private area on a website. It verifies that you are who you claim to be before granting access to personal data, dashboards, or restricted content.

2.1 What Happens When a User Clicks Login?

1. User enters email and password into the form.
2. The browser packages the data and sends it to the server (using POST).
3. The server looks up the email in the database.
4. The server compares the submitted password against the stored (hashed) password.
5. If they match — the server creates a session or token and sends it back.
6. The browser stores the session/token and redirects the user to their dashboard.
7. If they don't match — an error message is shown: "Invalid credentials."

2.2 Simple HTML Login Form

```
<!-- Login Form -->
<form id="loginForm" action="login.php" method="POST">

  <h2>Welcome Back</h2>

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" required
placeholder="you@example.com">

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required
placeholder=".....">

  <button type="submit">Login</button>
  <a href="forgot.html">Forgot password?</a>

</form>
```

2.3 JavaScript Login Validation

```
// Validate before sending to server
document.getElementById('loginForm').addEventListener('submit', function(e) {
  e.preventDefault(); // Stop form from reloading the page

  const email    = document.getElementById('email').value.trim();
  const password = document.getElementById('password').value.trim();

  if (!email || !password) {
    alert('Please fill in both fields.');
```

```
    return;
  }

  if (password.length < 8) {
    alert('Password must be at least 8 characters.');
```

```
    return;
  }

  this.submit(); // All good - proceed
});
```

IMPORTANT:

Frontend validation is a convenience for users — it catches mistakes early.
It is NOT a security measure. All real security checks happen on the SERVER.
A user can bypass JavaScript validation using browser developer tools.

3. Register Pages

A register (or sign-up) page creates a new user account. It collects identifying information, stores it securely in a database, and sets up the user's profile for future logins.

3.1 Common Registration Fields

- Full Name — identifies the user.
- Email Address — unique identifier and communication channel.
- Password — secret key to prove identity on future logins.
- Confirm Password — ensures the user typed the correct password.
- Phone Number (optional) — used for 2FA or account recovery.
- Date of Birth (optional) — age verification.
- Terms & Conditions checkbox — legal consent.

3.2 Simple HTML Registration Form

```
<form id="registerForm" action="register.php" method="POST">

  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" required>

  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="password" required>

  <label for="cpwd">Confirm Password:</label>
  <input type="password" id="cpwd" name="confirm_password" required>

  <button type="submit">Create Account</button>
</form>
```


3.3 JavaScript Registration Validation

```
document.getElementById('registerForm').addEventListener('submit', function(e) {
  e.preventDefault();

  const name  = document.getElementById('name').value.trim();
  const email = document.getElementById('email').value.trim();
  const pwd   = document.getElementById('pwd').value;
  const cpwd  = document.getElementById('cpwd').value;

  if (!name || !email || !pwd || !cpwd) {
    alert('All fields are required.');
```

```
    return;
  }
  if (pwd.length < 8) {
    alert('Password must be at least 8 characters.');
```

```
    return;
  }
  if (pwd !== cpwd) {
    alert('Passwords do not match.');
```

```
    return;
  }

  this.submit(); // Passed – send to server
});
```

WHY PASSWORDS MUST BE PROTECTED:

Passwords are NEVER stored as plain text in a real database.

The server runs the password through a hashing algorithm (e.g. bcrypt).

A hash is a one-way scramble — you cannot reverse it to get the original password.

Even if hackers steal the database, they only see scrambled hashes, not real passwords.

4. Authentication vs Authorization

These two words sound similar but describe completely different processes. Mixing them up is one of the most common mistakes in web development.

4.1 Authentication — "Who are you?"

Authentication is the process of verifying a user's identity. It answers: Are you really who you say you are?

Real-life analogy: Showing your national ID card at a bank. You prove your identity before being allowed inside.

- Happens at: Login page, Registration, Password reset.
- Evidence used: Email + password, fingerprint, OTP code, magic link.
- Result: A session or token is created — the system now knows who you are.

4.2 Authorization — "What can you access?"

Authorization happens AFTER authentication. It determines what permissions a verified user has — which pages, data, or actions they are allowed to use.

Real-life analogy: Being allowed inside the bank (authentication) but only permitted to go to the customer service area — not the vault (authorization).

- Happens at: Every protected request the user makes after login.
- Controlled by: User roles (admin, editor, student, customer).
- Result: Access granted or denied based on the user's role and permissions.

4.3 Comparison Table

Aspect	Authentication	Authorization
Question it answers	"Who are you?"	"What can you do?"
When it happens	At login / registration	After login, on every request
Evidence	Password, OTP, fingerprint	Role, permissions, access level
Example	User logs in with email + password	Admin can delete posts; users cannot
Failure result	"Invalid credentials — login failed"	"Access denied — 403 Forbidden"

5. Basic Website Security Concepts

Security must be thought about from the very first line of code. Even as a frontend developer, understanding these concepts will make you a much more professional and responsible developer.

5.1 Core Security Concepts

Concept	What It Means / Why It Matters
Strong Passwords	Passwords must be long, mix upper/lowercase, numbers, and symbols. Minimum 8 characters.
Password Hashing	Passwords are scrambled server-side (bcrypt, Argon2) before storage. Never stored as plain text.
HTTPS	Encrypts all data sent between the browser and server. Required for any site collecting user data.
Cookies	Small pieces of data stored in the browser. Used to remember sessions. Should be marked HttpOnly and Secure.
Sessions	Server-side storage of who is logged in. A session ID is stored in a cookie and matched server-side.
Tokens (JWT)	An encoded string stored in the browser (localStorage or cookie) proving the user is authenticated.
Input Validation	Checking user input is the correct type, length, and format — on both frontend and backend.
SQL Injection	Attackers insert malicious SQL into form fields to read or destroy the database. Always sanitize inputs server-side.
XSS (Cross-Site Scripting)	Attackers inject JavaScript into a page to steal data or hijack sessions. Never display raw user input as HTML.
Forgot Password	Should send a one-time secure link to the user's email — not display or email the original password.
Password Strength Meter	A frontend visual indicator that rates password strength in real-time as the user types.
Show/Hide Password	A toggle button on the password field (eye icon) that switches between type='password' and type='text'.

GOLDEN RULE FOR FRONTEND DEVELOPERS:

Frontend validation = USER EXPERIENCE (helps users fix mistakes quickly).

Backend validation = SECURITY (the only layer that actually protects your system).

NEVER trust data that comes from the browser. Always validate and sanitize on the server.

5.2 Show / Hide Password — Simple JS Example

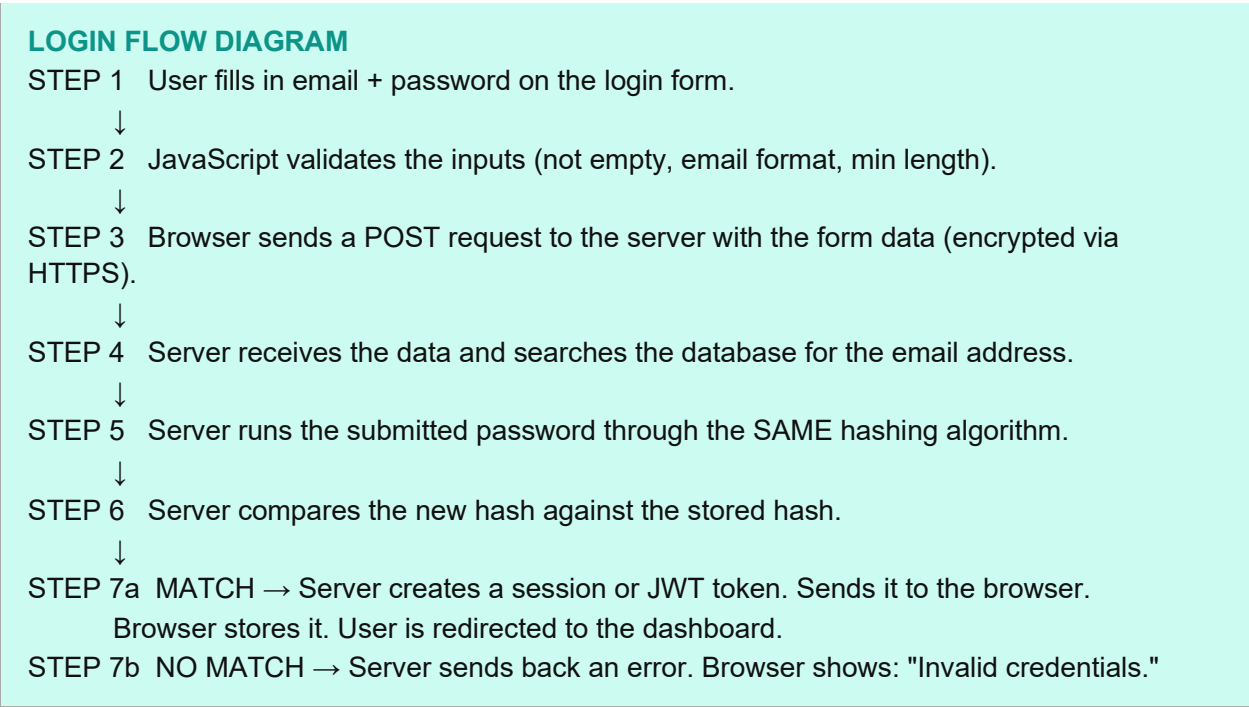
```
<!-- HTML -->
<input type="password" id="pwd">
<button type="button" onclick="togglePwd()">Show</button>

/* JavaScript */
function togglePwd() {
  const field = document.getElementById('pwd');
  const btn = event.target;
  if (field.type === 'password') {
    field.type = 'text';
    btn.textContent = 'Hide';
  } else {
    field.type = 'password';
    btn.textContent = 'Show';
  }
}
```

6. How Login Systems Work — Step by Step

Even though you are building the frontend, you need to understand the full login flow. This knowledge helps you design better forms, handle errors properly, and communicate correctly with backend developers.

6.1 The Full Login Flow



6.2 Sessions vs Tokens

Aspect	Sessions	Tokens (JWT)
What they are	A session ID stored in a cookie; data kept server-side.	A self-contained encoded string (JWT); stored client-side.
Where stored	Server memory or database.	Browser (localStorage or cookie).
When used	Traditional web apps (PHP websites).	Modern SPAs and API-based apps.
Logout process	Server deletes the session.	Browser deletes the token.

7. HTTP Methods

HTTP (HyperText Transfer Protocol) is the set of rules that governs communication between a browser and a server. HTTP methods tell the server WHAT ACTION to perform when it receives a request. Every form submission, page load, and API call uses an HTTP method.

7.1 GET — Request Data

GET is used to retrieve (read) data from a server. Data is appended to the URL as query parameters, making it visible in the browser's address bar.

- Use for: Search forms, viewing a product, loading a page.
- Data is visible in the URL — do NOT use for passwords or sensitive info.
- GET requests can be bookmarked and shared.

```
<!-- GET Form Example - Search -->
<form action="search.php" method="GET">
  <label for="q">Search:</label>
  <input type="text" id="q" name="q" placeholder="Type to search...">
  <button type="submit">Search</button>
</form>

<!-- After submit, the URL becomes: search.php?q=your+search+term -->
<!-- The server reads $_GET['q'] to process the search -->
```

7.2 POST — Send Data Securely

POST is used to send data to the server. Unlike GET, the data is inside the request body — not visible in the URL. POST is the correct method for any sensitive data such as login credentials, registration, or payments.

- Use for: Login forms, register forms, contact forms, file uploads.
- Data is hidden from the URL — safe for passwords and personal info.
- POST requests cannot be bookmarked.

```
<!-- POST Form Example - Login -->
<form action="login.php" method="POST">
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="password" required>

  <button type="submit">Login</button>
</form>

<!-- Data goes inside the request body - NOT in the URL -->
```

7.3 PUT — Update Existing Data

PUT replaces an entire existing record on the server with new data. It is commonly used in REST APIs when editing user profiles, updating a post, or replacing a product record.

- Example: User updates their full profile — name, email, bio, and avatar.
- The entire record is replaced with the new submission.
- HTML forms do not support PUT natively — it is used via JavaScript (fetch / XMLHttpRequest).

7.4 PATCH — Partial Update

PATCH is similar to PUT but only updates the specific fields that were changed — not the entire record.

- Example: User updates only their profile photo — other fields stay the same.
- More efficient than PUT for small edits.
- Also used via JavaScript — not directly available in HTML forms.

7.5 DELETE — Remove Data

DELETE is used to remove a record from the server. It is the most destructive method and should always require authentication and confirmation before executing.

- Example: User deletes their account, an admin removes a product or a post.
- Should always ask for confirmation: "Are you sure you want to delete this?"
- Also used via JavaScript — not available in HTML forms.

7.6 HTTP Methods Comparison Table

Method	Purpose	Real-World Example
GET	Read / retrieve data	Search form, view product, load page
POST	Create / send new data	Login, register, contact form, payment
PUT	Update — replace entire record	Edit full user profile
PATCH	Update — change partial record	Update only profile photo or username
DELETE	Remove a record	Delete account, remove a post
HTML FORMS ONLY SUPPORT GET AND POST. PUT, PATCH, and DELETE are used through JavaScript (fetch API or XMLHttpRequest). This is why frontend JavaScript skills are essential for building modern web applications.		

8. Frontend Validation Using JavaScript

Validation checks that user input is correct and complete before it is sent to the server. Done in JavaScript on the frontend, it provides instant feedback — making forms easier and more pleasant to use.

8.1 Empty Field Check

```
const name = document.getElementById('name').value.trim();

if (name === '') {
  showError('nameError', 'Name cannot be empty.');
```

return; // Stop here — do not submit

```
}
```

8.2 Email Format Validation

```
const email = document.getElementById('email').value.trim();
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

if (!emailPattern.test(email)) {

```
  showError('emailError', 'Please enter a valid email address.');
```

return;

```
}
```

8.3 Password Length Check

```
const pwd = document.getElementById('pwd').value;
```

if (pwd.length < 8) {

```
  showError('pwdError', 'Password must be at least 8 characters long.');
```

return;

```
}
```

8.4 Confirm Password Match

```
const pwd = document.getElementById('pwd').value;
const cpwd = document.getElementById('cpwd').value;
```

if (pwd !== cpwd) {

```
  showError('cpwdError', 'Passwords do not match. Please try again.');
```

return;

```
}
```


8.5 Reusable Error Display Function

```
// Call this anywhere to show an error message below a field
function showError(elementId, message) {
  const errorEl = document.getElementById(elementId);
  errorEl.textContent = message;    // Set the error text
  errorEl.style.color = 'red';      // Make it red
}

// In your HTML, add an empty <p> below each field:
// <p id="nameError" class="error"></p>
// <p id="emailError" class="error"></p>
```

9. Best Practices for Login & Register Pages

A well-built login or register page is not just functional — it is secure, accessible, and pleasant to use. Follow these practices on every form you build.

Best Practice	Why It Matters & How to Apply It
Clear, descriptive labels	Every field must have a visible label explaining exactly what to enter. Never rely on placeholder text alone.
Mobile-responsive design	Forms must work perfectly on small screens. Use 100% width inputs, adequate tap target sizes (min 44px height).
Friendly error messages	Say what is wrong AND how to fix it: "Password must be at least 8 characters" — not just "Error".
Show/hide password toggle	Add an eye icon to password fields. Users make fewer mistakes when they can verify what they typed.
Password strength indicator	A colour bar (red → yellow → green) that rates password strength in real-time as the user types.
Secure password requirements	Minimum 8 characters. Encourage mixing uppercase, lowercase, numbers, and symbols.
Use HTTPS always	Never deploy a login or register form on a plain HTTP site. All data must be encrypted in transit.
No sensitive info in URLs	Never put passwords, tokens, or user IDs in query strings. Use POST and keep them in the request body.
Accessible form design	Use <label for>, proper tab order, and ARIA attributes so screen readers can navigate the form.
Clear call-to-action buttons	"Login" is better than "Submit". "Create Account" is better than "OK". Be specific about what clicking does.
Link to registration & vice versa	Login page must link to the register page and the forgot password page — and register must link back.
Avoid exposing error details	Don't say "Email not found" — say "Invalid credentials" to avoid helping hackers narrow down valid emails.

10. Practical Mini Tasks

Complete the following three tasks. Each one builds on the previous. By the end you will have a working login and register page with validation, styled and ready to present.

Task 1 — Build a Login Form (HTML Only)

TASK INSTRUCTIONS:

Create a file called login.html.

Build a login form with the following fields:

- Email address input (type='email', required)
- Password input (type='password', required)
- A 'Login' submit button
- A link: 'Don't have an account? Register here'
- A link: 'Forgot your password?'

Use <label> for every input. Wrap the form in a centred card using basic CSS.

Set method='POST' on the form.

Add placeholder text to every input field.

Task 2 — Add JavaScript Validation to Your Login Form

TASK INSTRUCTIONS:

Create app.js and link it to your login.html.

Listen for the form's 'submit' event using addEventListener.

Call e.preventDefault() to stop the default page reload.

Validate the following and display an error message below the field (not alert()):

- Email field must not be empty.
- Email must match a valid email pattern (use a regex).
- Password must be at least 8 characters.

If all fields are valid, display a success message: 'Login successful! Redirecting...'

Bonus: Add a show/hide password toggle using the eye-icon technique from Section 5.

Task 3 — Design a Complete Registration Page

TASK INSTRUCTIONS:

Create register.html linked to the same app.js (or a new register.js).

Build a registration form with these fields:

- Full Name (text, required)
- Email Address (email, required)
- Password (password, min 8 chars, required)
- Confirm Password (password, must match, required)
- Checkbox: 'I agree to the Terms and Conditions' (required)

Add JavaScript validation for ALL fields including password match and checkbox.

Display a password strength indicator (weak / medium / strong) as the user types.

Style the form with CSS: card layout, brand colour, hover effects on the button.

Add a link at the bottom: 'Already have an account? Login here'

11. Quiz

Part A — Multiple Choice (Circle the correct answer)

1. Which HTTP method should be used for a login form that sends a password?
a) GET b) POST c) DELETE d) PUT
2. What does `e.preventDefault()` do in a JavaScript form event listener?
a) Submits the form immediately b) Stops the page from reloading
c) Clears all form fields d) Hides the form from view
3. Which of the following correctly describes Authentication?
a) Deciding what a user is allowed to access after login
b) Verifying who a user is (e.g. checking email + password)
c) Encrypting data sent over the internet
d) Storing user data in a database
4. A website stores passwords as plain text. Why is this a serious problem?
a) It uses too much database storage space
b) If the database is hacked, all passwords are immediately exposed
c) Plain text passwords take longer for the server to process
d) Browsers cannot read plain text passwords
5. Which HTTP method is used to update ONLY one field of an existing database record?
a) GET b) POST c) PUT d) PATCH
6. What is the purpose of the 'required' attribute on an HTML input element?
a) Sends the form data to the server automatically
b) Prevents the browser from submitting the form if that field is empty
c) Makes the field read-only so the user cannot change it
d) Adds a red border to the field automatically
7. What does HTTPS provide that plain HTTP does not?
a) Faster page loading speed
b) A free SSL certificate
c) Encryption of data sent between the browser and server
d) Access to a CDN (Content Delivery Network)
8. A user receives a 403 Forbidden error after logging in. Which concept does this relate to?
a) Authentication — the user is not logged in
b) Authorization — the user is logged in but not permitted to access that page
c) HTTP POST — the form method is incorrect
d) SQL Injection — the user is attempting an attack
9. Which element creates a dropdown list of options in an HTML form?
a) `<dropdown>` b) `<input type='list'>` c) `<select>` d) `<option>`

10. Why should a 'forgot password' system send a secure reset link rather than the original password?

- a) Sending the original password is faster for the user
- b) The original password is hashed and cannot be retrieved or emailed
- c) Reset links look more professional in emails
- d) All of the above

Part B — Short Answer

1. In your own words, explain the difference between Authentication and Authorization. Give one real-life example of each.
2. Explain why frontend JavaScript validation alone is not enough to secure a website. What must also happen on the backend?
3. List THREE HTTP methods, state what each one does, and give a real-world example of when each is used.
4. What is password hashing and why is it important? What would happen if passwords were stored in plain text?
5. Write the JavaScript code to check whether a password input field contains at least 8 characters, and display an error message below the field if it does not.

12. Chapter Summary

Well done on completing this training note! Here is a concise recap of every concept covered today.

Topic	Key Takeaway
Forms	HTML elements that collect user input and send it to a server. Common types: login, register, contact, search.
Form Elements	input, label, textarea, button, select, checkbox, radio. Always pair inputs with labels.
Login Pages	Verify a returning user's identity using email + password. Use method='POST'. Add JavaScript validation.
Register Pages	Collect new user details. Validate all fields. Confirm password match. Server hashes passwords before storage.
Authentication	"Who are you?" — verifying identity at login using credentials.
Authorization	"What can you access?" — controlling permissions after login based on user role.
Password Hashing	A one-way scramble of passwords server-side (bcrypt). Even if the DB is stolen, passwords stay safe.
HTTPS	Encrypts all browser-server communication. Required for any site handling user data.
Sessions & Tokens	Sessions store login state server-side; JWT tokens store it client-side. Both prove the user is logged in.
SQL Injection	Attackers exploit poorly sanitized inputs to manipulate databases. Always validate and sanitize server-side.
XSS	Attackers inject malicious JavaScript. Never render raw user input as HTML.
GET Method	Retrieves data. Sends params in the URL. Safe for searches, unsafe for passwords.
POST Method	Sends data in the request body. Hides it from the URL. Required for login, register, payments.
PUT Method	Replaces an entire existing resource. Used via JavaScript fetch().
PATCH Method	Updates part of an existing resource. Used via JavaScript fetch().
DELETE Method	Removes a resource. Should always require authentication + confirmation.
JS Validation	Catches mistakes early for better UX. Does NOT replace server-side security validation.

Topic	Key Takeaway
Show/Hide Password	Toggle between type='password' and type='text' using JavaScript — improves usability.
Best Practice: Errors	Show specific, friendly, actionable error messages below each field — not generic alert() popups.
Best Practice: Security	Never put sensitive data in URLs. Always use HTTPS. Never expose error details that help attackers.

REMEMBER:

As a frontend developer, your job is to build forms that are clear, accessible, validated, and secure in design.

The server and backend developer handle the actual authentication and data security.

But YOU must understand the full flow — so you can build the right forms, handle errors correctly,

and communicate effectively with the backend team.

Keep building. Keep learning. Keep growing.

Get Techy With Lucky | Tech Pulse Insider | Instructor: Lucky Nakola